

| 《数据库系统原理》实验报告（六）                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |         |    |     |    |           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----|-----|----|-----------|
| 题目：miniOB 进阶实验                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |         |    |     |    |           |
| 学号                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 2351136 | 姓名 | 李盛鹏 | 日期 | 2025.5.20 |
| <p>实验环境：</p> <p>软件环境：Windows 10 Docker MiniOB</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |         |    |     |    |           |
| <p>实验步骤及结果截图：</p> <p>1. 在 Docker 编译 miniob 的环境</p> <pre># cd miniob # cd build # cmake .. -- MiniOb/main found clang-format at /usr/bin/clang-format CMake Warning at clang_check.cmake:29 (message):   MiniOb/main couldn't find clang-tidy. Call Stack (most recent call first):   CMakeLists.txt:1 (include)  -- MiniOb/main found cpplint at /root/miniob/build_support/cpplint.py -- This is Project source dir /root/miniob -- This is PROJECT_BINARY_DIR dir /root/miniob/build -- HOME dir: /root -- This is UNIX -- Disable debug -- CMAKE_CXX_COMPILER_ID is GNU -- CMAKE_INSTALL_PREFIX has been set as /usr/local  [ 95%] Built target md5_test [ 96%] Built target mem_pool_test [ 97%] Built target persist_test [ 98%] Built target pidfile_test [ 99%] Built target record_manager_test [100%] Built target ring_buffer_test # ./bin/observer -s miniob.sock -f ../etc/observer.ini &amp; # Successfully load ../etc/observer.ini</pre> <p>实验步骤及结果截图：</p> <p>Drop table:</p> <p>1.解析 DROP TABLE 语句：</p> <p>在执行删除操作前，需对用户输入的 DROP TABLE 语句进行语法解析，将其转化为内部可识别的结构形式（语法解析树）。识别形如 DROP TABLE 表名 的语句。解析器在识别到该语句结构后，将创建一个新的 ParsedSqlNode 对象，并将其类型设置为 SCF_DROP_TABLE，表示这是一个删除表的操作。同时，从语句中提取出表名 ID（即 \$3），存储到 drop_table.relation_name 字段中供后续处理使用。最后释放 \$3 所占内存，防止内存泄漏。</p> <pre>drop_table_stmt:    /*drop table 语句的语法解析树*/ DROP TABLE ID {     \$\$ = new ParsedSqlNode(SCF_DROP_TABLE);     \$\$-&gt;drop_table.relation_name = \$3;     free(\$3); };</pre> <p>2. 构建 DropTable 语义对象：</p> |         |    |     |    |           |

在完成 SQL 语法规解析并生成解析树后，需进一步将其转换为执行阶段能够理解和处理的语义对象。此时通过 `DropTableStmt::create` 函数，将解析阶段获得的表名信息封装进一个专门用于表示“删除表”语义的对象 `DropTableStmt` 中，并赋值给通用语义指针 `stmt`，供后续执行器使用。同时通过日志函数输出待删除的表名，以便调试记录。函数处理成功后返回执行成功标志，表示语义对象创建完成。

```
#include "sql/stmt/drop_table_stmt.h"
#include "event/sql_debug.h"

RC DropTableStmt::create(Db *db, const DropTableSqlNode &drop_table, Stmt *&stmt)
{
    stmt = new DropTableStmt(drop_table.relation_name);
    sql_debug("drop table statement: table name %s", drop_table.relation_name.c_str());
    return RC::SUCCESS;
}
```

### 3. 执行 Drop Table 操作:

在语义对象创建完成后，执行阶段需要根据该语义对象实际执行删除操作。此时执行器会从 SQL 事件中获取通用语义对象指针，并断言该语句类型确实是 `DROP TABLE`，确保语义正确无误。随后将其转换为具体的 `DropTableStmt` 类型，从中提取出要删除的表名。接着通过当前会话对象获取当前使用的数据库，并调用数据库的 `drop_table` 方法来删除指定表。最终返回删除操作的结果码，以指示是否执行成功。

```
25 RC DropTableExecutor::execute(SQLStageEvent *sql_event)
26 {
27     Stmt *stmt = sql_event->stmt();
28     Session *session = sql_event->session_event()->session();
29     ASSERT(stmt->type() == StmtType::DROP_TABLE,
30           "drop table executor can not run this command: %d", static_cast<int>(stmt->type()));
31
32     DropTableStmt *drop_table_stmt = static_cast<DropTableStmt *>(stmt);
33
34
35     const char *table_name = drop_table_stmt->table_name().c_str();
36     RC rc = session->get_current_db()->drop_table(table_name);
37     return rc;
38 }
39 }
```

### 4. 默认处理器中的 Drop Table 接口定义:

为了实现完整的 `DROP TABLE` 功能，系统在处理器模块中定义了统一的接口函数用于执行删除表操作。在 `DefaultHandler` 类中，定义了接收数据库名与表名的 `drop_table` 函数接口，为后续具体数据库操作提供入口。但当前该函数尚未实现，返回值为“未实现”状态，表示该功能尚待补全。此接口作为系统功能扩展的一部分，需由具体数据库实例进行重写和实现，以完成实际的表删除逻辑。

```
RC DefaultHandler::drop_table(const char *dbname, const char *relation_name)
{
    return RC::UNIMPLENMENT;
}
```

### 5. 在数据库中执行 Drop Table 的核心操作:

当执行器发出删除表的请求后，数据库对象会接收到具体的表名，并在其内部维护的表管理结构中查找该表是否存在。若查找失败，即表未打开或不存在，则记录警告日志，并返回“表不存在”的错误码。若表存在，则调用该表的 `drop` 方法，执行实际的物理删除操作（例如删除文件）。若删除失败，则记录错误并返回对应错误码；若成功，则从已打开表列表中移除该表，并释放其占用的内存资源。最终返回整个删除过程的状态码，供上层判断是否成功完成操作。

```
RC Db::drop_table(const char *table_name)
{
    RC rc = RC::SUCCESS;
    Table * table = nullptr;
    auto it = opened_tables_.find(table_name);
    if(it == opened_tables_.end())
    {
        LOG_WARN("table : %s not exist", table_name);
        rc = RC::SCHEMA_TABLE_NOT_EXIST;
    }
    else if( (table = it->second) == nullptr)
    {
        LOG_WARN("table : %s not exist", table_name);
        rc = RC::SCHEMA_TABLE_NOT_EXIST;
    }
    else if((rc = table->drop(path_.c_str())) != RC::SUCCESS)
    {
        LOG_WARN("table drop file,errno: %s", strrc(rc));
    }
    else
    {
        opened_tables_.erase(it);
        delete table;
    }
    return rc;
}
```

Select table:

#### 1. 解析 SELECT 语句:

在执行查询操作前，数据库需要先对输入的 SELECT 语句进行语法解析。语法解析器通过 Bison 所定义的规则识别出 SELECT column1, column2 FROM 表名 WHERE ... 等结构，将查询的列、目标表名、过滤条件等信息解析成语法树节点，并封装为 SelectSqlNode 类型的数据结构，为后续语义分析做准备。

```

select_stmt: /* select 语句的语法解析树 */
SELECT expression_list
{
    $$ = new ParsedSqlNode(SCF_SELECT);
    if ($2 != nullptr) {
        std::reverse($2->begin(), $2->end());
        $$->selection.project_exprs.swap(*$2);
        delete $2;
    }
}
| SELECT expression_list FROM from_node from_list where opt_group_by opt_having opt_order_by
{
    $$ = new ParsedSqlNode(SCF_SELECT);
    if ($2 != nullptr) {
        std::reverse($2->begin(), $2->end());
        $$->selection.project_exprs.swap(*$2);
        delete $2;
    }
    if ($5 != nullptr) {
        $$->selection.relations.swap(*$5);
        delete $5;
    }
    $$->selection.relations.push_back(*$4);
    std::reverse($$->selection.relations.begin(), $$->selection.relations.end());

    $$->selection.conditions = nullptr;
    if ($6 != nullptr) {
        $$->selection.conditions = $6;
    }
    if ($7 != nullptr) {
        $$->selection.groupby_exprs.swap(*$7);
        delete $7;
        std::reverse($$->selection.groupby_exprs.begin(), $$->selection.groupby_exprs.end());
    }
    $$->selection.having_conditions = nullptr;
    if ($8 != nullptr) {
        $$->selection.having_conditions = $8;
    }

    if ($9 != nullptr) {
        $$->selection.orderbys.swap(*$9);
        delete $9;
    }
    delete $4;
}
;
    
```

## 2. 构建 Select 语义对象:

解析完成后，系统调用 `SelectStmt::create` 函数，根据解析树构建语义对象。此过程中会校验表名是否存在，字段名是否合法，WHERE 子句中的表达式是否有效，并提取所有必要的查询元信息（如表结构、列偏移等），最终形成一个可被执行器理解的 `SelectStmt` 对象，准确描述本次查询的逻辑含义。

```

switch (stmt->type()) {
case StmtType::SELECT: {
    ProjectPhysicalOperator *project_operator = static_cast<ProjectPhysicalOperator *>(physical_operator.get());
    for (const unique_ptr<Expression> & expr : project_operator->projections()) {
        if (expr->alias().empty()) {
            schema.append_cell(expr->name().c_str());
        } else {
            schema.append_cell(expr->alias().c_str());
        }
    }
} break;
    
```

## 3. 执行 SELECT 操作:

执行器接收到 `SelectStmt` 语义对象后，会依照其中提供的查询条件，生成扫描表的计划。执行阶段会调用表对象的扫描接口，根据是否存在 WHERE 条件决定使用全表扫描还是条件过滤。对于多表查询，还可能触发连接算法或子查询处理逻辑。

## 4. 读取数据并处理:

存储层通过表的 `Trx::scan_record` 接口逐条读取记录，将记录中的列值映射为结果行。若查询中包含过滤条件（WHERE），则对每条记录进行条件判断，仅保留满足条件的记录；若包含聚合（如 COUNT、SUM）或分组（GROUP BY），则额外构建分组哈希表或累计器进行聚合运算。

## 5. 排序、分页与返回结果:

若语句中包含 **ORDER BY** 子句，执行器会对结果集按指定列排序；若存在 **LIMIT**，则对结果集进行截断，仅返回指定数量的行。最终将处理完成的结果封装为 **TupleSet**，由 **SqlResult** 统一格式化并返回给客户端展示。

执行测试样例示例的实验结果截图：

#### Drop\_table 测试样例

```
# ./bin/obclient -s miniob.sock
miniob > create table t(id int, age int);
SUCCESS
miniob > create table t(id int, name char);
FAILURE
miniob > drop table t;
SUCCESS
miniob > create table t(id int, name char);
SUCCESS
```

#### Select\_table 测试样例

```
miniob > create table t(id int, name char);
SUCCESS
miniob > drop table t
SUCCESS
miniob > CREATE TABLE student(id INT, name CHAR(20), score FLOAT);
SUCCESS
miniob > INSERT INTO student VALUES(1, 'Alice', 90.5);
SUCCESS
miniob > SELECT * FROM student;
id | name | score
1 | Alice | 90.5
miniob > SELECT name, score FROM student;
name | score
Alice | 90.5
miniob > SELECT * FROM student WHERE score > 80;
id | name | score
1 | Alice | 90.5
```

## drop-table

通过率：76.6%      难度：简单      题目总分：10

1. 删除表，清除表相关的资源。
2. 要删除所有与表关联的数据，不仅包括磁盘中的文件，还包括内存中的索引等数据。

测试用例示例：

```
create table t(id int, age int);
create table t(id int, name char);
drop table t;
create table t(id int, name char);
```

select-tables

通过率: 42.4%    难度: 中等    题目总分: 10

- 1. 当前系统支持单表查询的功能，需要在此基础上支持多张表的笛卡尔积关联查询，并考虑基本类型转换。
- 2. 需要实现select \* from t1,t2; select t1.\*,t2.\* from t1,t2;以及select t1.id,t2.id from t1,t2;查询可能会带条件。
- 3. 查询结果展示格式参考单表查询。每一列必须带有表信息，比如:

t1.id | t2.id

1 | 1

测试用例示例:

```
select * from t1,t2;
select * from t1,t2 where t1.id=t2.id and t1.age > 10;
select * from t1,t2,t3;
```

|               |    |    |
|---------------|----|----|
| drop-table    | 10 | 10 |
| select-tables | 10 | 10 |